# Cours de C++

BTS CIEL-IR — Visual Studio Code, g++, norme C++17

# Du code source à l'exécutable

1. **Éditeur**
   Visual Studio Code : écrire le code source (.cpp)

2. **Compilateur**
   g++ : traduire le code en exécutable

3. **Débogueur**
   exécuter pas à pas pour trouver les erreurs

```cpp
#include <iostream>

int main() {
    std::cout << "Bonjour !" << '\n';
    return 0;
}
```

```
g++ -std=c++17 -Wall -Wextra prog.cpp -o prog
./prog
```

*Après chaque modification : recompiler.*

# Erreur, avertissement ou bug ?

### 1 Erreur de compilation

Aucun exécutable produit.
Ex. : point-virgule oublié, #include manquant.

### 2 Avertissement

Exécutable produit, mais risque signalé.
Ex. : variable utilisée sans valeur.

### 3 Erreur d'exécution

Le programme plante ou donne un résultat faux.
Se détecte par les tests.

**Règle :** un avertissement signale presque toujours un vrai problème. On compile avec -Wall -Wextra et on corrige tout.

# Variables, types et constantes

| Type | Contenu | Exemple |
|---|---|---|
| bool | vrai / faux | `true` |
| char | un caractère (1 octet) | `'A'` |
| int | entier signé | `-25` |
| double | nombre réel | `12.5` |
| std::string | chaîne de caractères | `"Bonjour"` |

```
int age{20};
double temperature{21.5};
int compteur{};      // 0
// int v{3.7};      // refuse
constexpr int MAX{100};
```

**Toujours initialiser**

Lire une variable non initialisée est un comportement indéfini.

# Division entière ou réelle ?

`7 / 3`

**2**

division entière

`7 % 3`

**1**

reste

`static_cast<double>(7) / 3`

**2.33333**

division réelle

- Entre deux entiers, / est une division entière : la partie décimale est perdue.
- static_cast<double>(7 / 3) vaut 2 : la division est faite avant la conversion.
- **Division par zéro et dépassement d'un entier signé : comportement indéfini.**

# Comparer et combiner des conditions

| Opérateur | Vrai si… |
|---|---|
| `a && b` | les deux sont vraies |
| `a \|\| b` | au moins une est vraie |
| `!a` | a est fausse |

**! Piège : = ou == ?**

if (x = 5) affecte 5 et reste toujours vrai.
Comparer : if (x == 5).

# if, else if et switch

```
if (t < 0) {
    std::cout << "Negative\n";
} else if (t <= 30) {
    std::cout << "Normale\n";
} else {
    std::cout << "Elevee\n";
}
```

Conditions testées dans l'ordre ; accolades toujours.

```
switch (choix) {
    case 1:
        demarrer();
        break;
    default:
        erreur();
        break;
}
```

Expression entière, case constants, un break par cas.

# Trois boucles, trois usages

### 1 for

Nombre de répétitions connu, parcours d'une collection

```
for (int i = 0; i < n; i++)
```

### 2 while

Répéter tant qu'une condition est vraie

```
while (compteur <= 5)
```

### 3 do … while

Exécuter au moins une fois (menu, saisie)

```
do { … } while (choix != 0);
```

*Sans mise à jour de la condition : boucle infinie.*

# Tableau, std::array ou std::vector ?

| | Tableau C | std::array | std::vector |
|---|---|---|---|
| Taille | fixe | fixe | variable |
| Connaît sa taille | non | `size()` | `size()` |
| Indice vérifié | non | `at(i)` | `at(i)` |
| Usage | simple, Arduino | 4 octets IPv4 | mesures saisies |

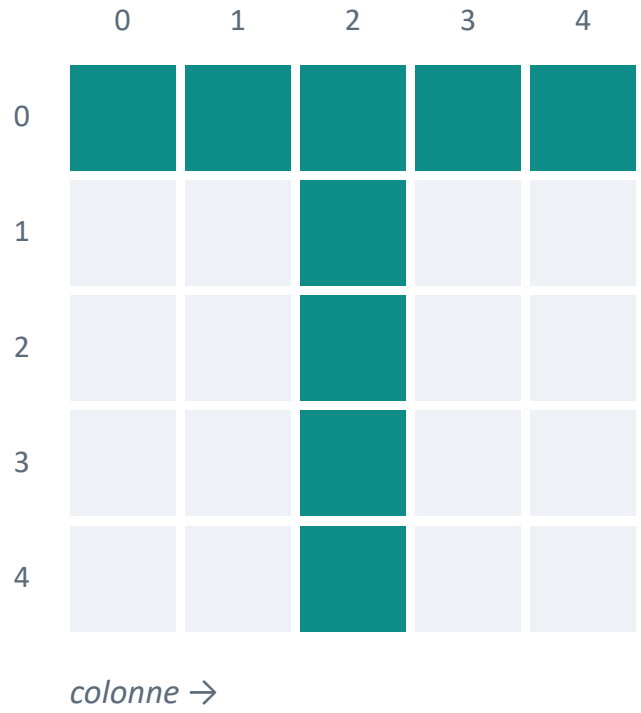
```
std::vector<double> m{};
m.push_back(21.5);
m.push_back(23.5);

for (double v : m) {
    somme += v;
}
```

**Indices de 0 à taille − 1**

Hors limites : comportement indéfini.

# Tableaux à deux dimensions




```
int pixels[5][5]{ … };

for (int ligne = 0; ligne < 5; ligne++) {
    for (int col = 0; col < 5; col++) {
        // pixels[ligne][col]
    }
}
```

- Premier indice : la ligne ; second : la colonne.
- pixels[1][2] vaut 1 (ligne 1, colonne 2).

# Fonctions et passage des paramètres

```
double moyenne(double x, double y);
```

**type de retour** · nom · paramètres = la signature

| Paramètre | La fonction… |
|---|---|
| `double x` | reçoit une copie |
| `double& x` | peut modifier l'original |

**! Ordre des arguments**

Pour regler(int taille, int vitesse), l'appel regler(1, 16) donne taille = 1.

# Séparer le code en modules

**`conversion.h`**
déclarations
prototypes, constexpr

→

**`conversion.cpp`**
définitions
des fonctions

→

**`main.cpp`**
#include "conversion.h"
utilise le module

| Message | Étape | Cause probable |
|---|---|---|
| `was not declared` | compilation | en-tête non inclus |
| `undefined reference` | édition de liens | .cpp oublié dans la commande |
| `multiple definition` | édition de liens | fonction définie dans un .h |

```
g++ -std=c++17 -Wall -Wextra main.cpp conversion.cpp -o conversion
```

# Tester une fonction

**Cas de test**
entrée choisie

**Attendu**
d'après le cahier des charges

**Obtenu**
renvoyé par la fonction

**Verdict**
OK si égaux, sinon ÉCHEC

## Valeurs limites pour une plage 0 … 100 incluses




*Si le code teste valeur < 100 au lieu de <= 100, seul le test de la borne 100 échoue.*

# Binaire, hexadécimal et ASCII

| `156` | `1001 1100` | `0x9C` |
|---|---|---|
| décimal | binaire | hexadécimal |

*= une seule et même valeur*

- 1 chiffre hexadécimal = 4 bits
- 1 octet = 2 chiffres hexadécimaux
- std::hex, std::bitset<8>, std::stoul

| Caractère | Code |
|---|---|
| `'0' … '9'` | `0x30 … 0x39` |
| `'A' … 'Z'` | `0x41 … 0x5A` |

# Opérateurs bit à bit et masques

| Opération sur le bit n | Écriture |
|---|---|
| Tester | `(etat & (1u << n)) != 0` |
| Activer | `etat \|= (1u << n);` |
| Désactiver | `etat &= ~(1u << n);` |
| Inverser | `etat ^= (1u << n);` |
| Extraire un octet | `(v >> 8) & 0xFF` |

**Logique ≠ bit à bit**

&&, ||, ! combinent des conditions.
&, |, ^, ~ transforment des bits.

# Applications : IPv4 et masquage XOR

```
Adresse : 192.168.10.77
Masque  : 255.255.255.192 (/26)
ET      : 192.168.10.64  -> reseau
reseau | ~masque
        : 192.168.10.127 -> diffusion
```

**Adresse réseau = adresse & masque**

```
for (i = 0; i < n; i++) {
    sortie[i] = message[i]
                ^ cle[i % 4];
}
```

**Clé répétée : indice i % taille**

*Un « 14e octet » dans une documentation correspond à l'indice 13 en C++.*